Implementation and Challenges

For this assignment, I implemented an exact Dynamic Programming algorithm for makespan scheduling on identical machines. My approach recursively assigns jobs to machines, using memoisation and sorted machine-load tuples to avoid repeatedly solving equivalent subproblems.

The algorithm's worst-case behaviour occurs when many distinct machine-load configurations must be explored and the theoretical lower bound cannot be reached. Inputs containing varied processing times can be particularly challenging because they produce fewer equivalent states than repeated-size jobs.

On larger instances, this limitation can contribute to exponential growth in the number of states explored.

Evaluation and Learning

The most challenging design decision was determining how to reduce redundant computations without compromising optimality.

Initially, I explored subset-based DP using bitmasking. However, its (O(m3^n)) time complexity made it unattractive for larger job counts. I instead chose a top-down approach where each state is represented by dp(job_index, loads).

A key difficulty was recognising that machine identities do not matter when all machines are identical. For example, load configurations (3, 5) and (5, 3) represent equivalent scheduling problems. By sorting the loads before each recursive call, I could represent both configurations using a single cached state.

However, this introduced another consideration: sorting removes machine identities, making it harder to reconstruct actual job assignments. I addressed this by separating the optimisation calculation from schedule reconstruction, using cached DP results to identify assignments that achieve the optimal makespan.

My experiments also challenged my assumptions about optimisation. Some techniques reduced redundant operations but did not necessarily improve execution time because their additional overhead outweighed the savings.

Future Considerations

If I implemented the algorithm again, I would investigate job ordering earlier in development. My current implementation processes jobs in their supplied order, but assigning larger jobs first can help the search discover good solutions sooner, particularly when lower-bound early stopping is enabled.

I would develop a more systematic benchmarking and testing framework from the beginning, benchmarking and testing during development rather than after, which led to code refactoring.

Overall, this assignment changed how I approach exact algorithms. I learned that efficiency is not about reducing the number of operations, but about selecting effective state representations, exploiting mathematical properties and validating optimisations through experimentation.

What I now understand

I understand how important state representation in DP is. By recognising that identical machines are interchangeable, I learned how sorting machine loads allows for configurations that share the same cached state without affecting optimality. Furthermore, I developed a better understanding of lower-bound early stopping and how mathematical guarantees can reduce unnecessary computation. Overall, I learned that DP requires precise optimisation and correctness.